Impact of Feature Preprocessing on Classical Machine Learning Models for Phishing URL Detection

Baris Kaban Goktug Yildirim Mert Arslan
Vrije Universiteit Amsterdam

23/03/2026

Abstract

Feature preprocessing is standard practice in machine learning, but how much it actually matters depends on the model and the data. We investigate this question by evaluating five preprocessing strategies (no preprocessing, min-max scaling, standardisation, PCA, and whitening) across five classical classifiers on a phishing URL dataset with over 235,000 samples and 50 numerical features. Using stratified five-fold cross-validation and paired t-tests, we find that most models achieve near-perfect performance regardless of preprocessing. The RBF-SVM tells a different story: without scaling its ROC-AUC sits at 0.997, and a controlled scale-distortion experiment pushes it down to 0.532, barely above random chance ($p < 0.001$). Any scaling method fully restores it. We also find that k-NN benefits from standardisation but not from min-max scaling, that Naïve Bayes is harmed by PCA, and that the ranking of important features changes entirely depending on whether the data is scaled.

1 Introduction

In the PhiUSIIL phishing URL dataset, the feature with the highest variance differs from the feature with the lowest variance by a factor of more than 10^{15}. For any model that computes distances between data points, this means one feature will dominate everything else. Whether that matters in practice, and how much, is the question this paper addresses.

Many classical machine learning algorithms assume that input features are represented on comparable numerical scales. Algorithms that rely on distance computations, such as k-Nearest Neighbours (k-NN), or on kernel evaluations, such as Support Vector Machines with a radial basis function (RBF) kernel, are particularly affected: the large-range feature dominates the distance computation and may mask informative but small-range variables (Hastie et al., 2009). Preprocessing techniques such as min-max scaling, standardisation, PCA, and whitening are widely recommended to address this (Jolliffe, 2002; Bishop, 2006), but their practical impact depends on both the dataset and the model family (Singh & Singh, 2020; Fernandes et al., 2022; Ahsan et al., 2021).

We evaluate five preprocessing strategies across five model variants on the PhiUSIIL phishing URL dataset (Prasad & Chandra, 2024), comprising over 235,000 labelled samples and 50 numerical features. Beyond measuring performance, we apply paired statistical tests to determine which differences are significant, we conduct a controlled scale-distortion experiment to isolate the effect of feature magnitude, and we analyse how preprocessing alters the apparent importance of individual features. We also implement min-max scaling, standardisation, and k-NN from first principles to verify our understanding of the underlying computations.

2 Related Work

The theoretical sensitivity of distance-based classifiers to feature scaling is well-understood. Cover & Hart (1967) introduced k-NN, whose predictions depend directly on distances in the feature space; unscaled features with large ranges therefore receive disproportionate influence. Cortes & Vapnik (1995) showed that the RBF kernel in SVMs depends on $\|\mathbf{x}_i - \mathbf{x}_j\|^2$, making it scale-sensitive by design. Hsu et al. (2003) provide practical guidance for SVM users, explicitly recommending feature scaling before

training to avoid features with larger ranges dominating the kernel computation. Their guide specifically notes that failing to scale can lead to features with larger numerical ranges dominating the optimisation, an effect we observe directly in our RBF-SVM results. Linear models such as Logistic Regression (Bishop, 2006) can absorb scale differences into their weight vectors during optimisation, which explains their empirical robustness.

Several empirical studies have confirmed these theoretical expectations on real data. Singh & Singh (2020) evaluated multiple normalisation techniques across a range of classifiers and found that scaling significantly improves accuracy for distance-based models, though their analysis focused on accuracy alone and did not test whether the observed differences were statistically significant. Fernandes et al. (2022) demonstrated statistical significance of scaler choice across 82 datasets with 20 classification algorithms, though their analysis did not isolate the interaction between scaler type and distance metric, a gap our k-NN results begin to address. Ahsan et al. (2021) examined six scaling methods on eleven algorithms for a medical dataset, reaching broadly similar conclusions.

Dimensionality reduction via PCA (Jolliffe, 2002) and PCA-whitening can additionally decorrelate features and concentrate predictive information into fewer components. Bishop (2006) discusses how whitening transforms the covariance matrix to the identity, which should theoretically benefit models that assume feature independence, such as Naïve Bayes (Domingos & Pazzani, 1997). Whether this theoretical advantage holds in practice is less clear, and our results suggest it does not always.

What is missing from this body of work is a controlled experiment that isolates preprocessing as the sole variable, separating its role as a performance booster from its role as a safeguard against scale-related failure. That is what we attempt here, combining preprocessing comparison with statistical testing, a controlled distortion experiment, and feature importance analysis on a recent, large-scale phishing detection dataset.

3 Dataset

We use the PhiUSIIL Phishing URL dataset (Prasad & Chandra, 2024), obtained from the UCI Machine Learning Repository. It contains 235,795 URLs labelled as either legitimate (class 1, 57%) or phishing (class 0, 43%), represented by 50 numerical features derived from URL structure, lexical properties, and webpage metadata. Non-numeric columns were excluded prior to the experiments.

An exploratory analysis reveals substantial differences in feature scales. The variance ratio between the highest-variance feature (`LargestLineLength`, $\sigma^2 \approx 2.3 \times 10^{10}$) and the lowest (`ObfuscationRatio`, $\sigma^2 \approx 1.5 \times 10^{-5}$) exceeds 10^{15}. This extreme ratio motivates the use of feature scaling, particularly for models that compute distances. A correlation analysis further shows moderate linear dependence between several feature pairs, suggesting that decorrelation through PCA or whitening may reduce redundancy. A PCA on the standardised features shows that 36 out of 50 components are needed to retain 95% of the total variance, indicating that the effective dimensionality of the dataset is only moderately lower than the original.

The dataset's high separability likely stems from features such as `URLSimilarityIndex`, which takes the value 100 for nearly all legitimate URLs but varies widely for phishing URLs, creating a near-perfect decision boundary for most classifiers. We verified that this separability is not an artefact of data leakage by running a label-shuffle test (ROC-AUC = 0.499, confirming that the signal disappears when labels are randomised) and by evaluating with domain-aware group k-fold cross-validation, which produced identical results. The class-conditional feature distributions (Appendix B, Figure 7) reveal that several features show near-complete separation between legitimate and phishing URLs. `URLSimilarityIndex` clusters near 100 for legitimate URLs but spreads widely for phishing URLs, while `IsHTTPS` shows a strong binary contrast. The correlation heatmap (Appendix B, Figure 5) confirms moderate dependence between URL length features, suggesting some redundancy that PCA might be expected to exploit.

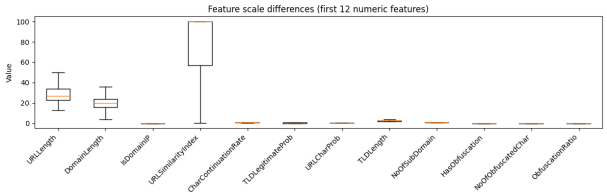

Figure 1: Feature scale differences across selected numeric features. URLSimilarityIndex spans 0–100 while most other features cluster near zero, illustrating the extreme variance ratio that motivates preprocessing.

4 Methods

4.1 Preprocessing Techniques

We evaluate five preprocessing strategies.

None. Features are used without transformation, serving as a baseline.

Min-max scaling. Each feature x_j is rescaled to $[0, 1]$:

$$x'_j = \frac{x_j - \min(x_j)}{\max(x_j) - \min(x_j)} \tag{1}$$

Standardisation. Each feature is shifted to zero mean and unit variance:

$$x'_j = \frac{x_j - \mu_j}{\sigma_j} \tag{2}$$

where μ_j and σ_j are the mean and standard deviation of feature j, computed on the training fold only.

PCA. After standardisation, we apply PCA and retain the smallest number of components that explain at least 95% of the total variance. Given the data matrix $\mathbf{X}$, PCA finds an orthogonal basis of principal components $\mathbf{c}_1, \ldots, \mathbf{c}_m$ that maximise the variance of the projected data (Jolliffe, 2002).

Whitening. Like PCA, but the retained components are additionally scaled to unit variance, producing uncorrelated features with identical spread. This is equivalent to transforming the data so that its covariance matrix becomes the identity (Bishop, 2006).

4.2 Machine Learning Models

Logistic Regression models the posterior probability $P(y = 1 \mid \mathbf{x})$ as $\sigma(\mathbf{w}^\top \mathbf{x} + b)$, where $\sigma(z) = 1/(1 + e^{-z})$ is the logistic function. Training minimises the regularised negative log-likelihood:

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 + C \sum_{i=1}^{n} \log\big(1 + e^{-y_i(\mathbf{w}^\top \mathbf{x}_i + b)}\big) \tag{3}$$

where C controls the trade-off between regularisation strength and data fit. Because each weight w_j can compensate for the scale of the corresponding feature x_j, the optimiser implicitly rescales the inputs, making the model robust to feature magnitude.

k-Nearest Neighbours classifies a test instance by a majority vote among its k closest training instances, measured by a distance metric $d(\mathbf{x}_i, \mathbf{x}_j)$. For the Euclidean metric, $d(\mathbf{x}_i, \mathbf{x}_j) = \sqrt{\sum_{j=1}^{m}(x_{ij} - x_{kj})^2}$: a feature with variance 10^{10} will contribute roughly 10^{10} times more to this sum than a feature with variance 1. This makes k-NN inherently sensitive to feature scale (Cover & Hart, 1967).

Support Vector Machines find the separating hyperplane $\mathbf{w}^\top \mathbf{x} + b = 0$ that maximises the margin between classes by solving the soft-margin optimisation problem:

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \frac{1}{2}\|\mathbf{w}\|^2 + C \sum_{i=1}^{n} \xi_i \tag{4}$$

subject to $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i$ and $\xi_i \geq 0$, where ξ_i are slack variables that allow misclassifications and C penalises them (Cortes & Vapnik, 1995). For non-linear boundaries, the input is implicitly mapped to a higher-dimensional space via a kernel function. We evaluate the RBF kernel $K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma\|\mathbf{x}_i - \mathbf{x}_j\|^2)$, which depends directly on the squared Euclidean distance and is therefore sensitive to feature scale by the same mechanism as k-NN.

Gaussian Naïve Bayes estimates the posterior $P(y \mid \mathbf{x}) \propto P(y) \prod_{j=1}^{m} P(x_j \mid y)$ by assuming that features are conditionally independent given the class, modelling each $P(x_j \mid y)$ with a Gaussian $\mathcal{N}(\mu_{jy}, \sigma^2_{jy})$ (Domingos & Pazzani, 1997). Since each feature is modelled independently with its own mean and variance, the algorithm naturally accounts for differing scales.

4.3 Hyperparameter Selection

Hyperparameters were selected by grid search with cross-validation (Kohavi, 1995) on the training data only, preventing information leakage into the test folds. Although randomised search can be more efficient for large

search spaces (Bergstra & Bengio, 2012), we used exhaustive grid search given the manageable number of hyperparameter combinations. The search spaces were: k-NN: $k \in \{3, 5, 7, 10, 15, 20, 30\}$, metric $\in$ {Euclidean, Manhattan}; RBF-SVM: $C \in \{0.1, 1, 10, 100\}$, $\gamma \in$ {scale, auto, 0.01, 0.001}; Logistic Regression: $C \in \{0.001, 0.01, 0.1, 1, 10, 100\}$; Naïve Bayes: variance smoothing $\in \{10^{-12}, \ldots, 10^{-5}\}$. The best-performing configuration on standardised data was used for all subsequent experiments.

4.4 Baseline Classifiers

To contextualise the results we include two baseline classifiers: a *majority-class* classifier that always predicts the most frequent label, and a *stratified random* classifier that predicts according to the class prior. These establish a lower bound on meaningful performance.

4.5 From-Scratch Implementations

To demonstrate our understanding of the preprocessing and classification methods, we implemented min-max scaling, standardisation, and k-NN using only basic numerical operations. Each implementation was verified against established library routines on a subset of the data; we obtained exact agreement (maximum absolute difference $< 10^{-15}$) for both scalers and 100% prediction agreement for k-NN.

4.6 Experimental Pipeline

All experiments use stratified five-fold cross-validation on a representative subsample of 50,000 instances drawn from the full dataset to ensure computational feasibility, particularly for the SVM models. All models and preprocessing steps were implemented using established machine learning libraries (Pedregosa et al., 2011). Preprocessing transformations are fitted exclusively on the training portion of each fold to prevent data leakage. Performance is measured by ROC-AUC, balanced accuracy, and F1 score for the positive class; stability is assessed via standard deviation across folds. Differences between preprocessing variants are tested for statistical significance using a paired t-test across fold-level scores, with a significance threshold of $\alpha = 0.05$.

5 Results

5.1 Hyperparameter Tuning

Grid search selected the following parameters: k-NN with $k = 20$ and Manhattan distance; RBF-SVM with $C = 100$ and $\gamma = 0.001$; Logistic Regression with $C = 10$; Naïve Bayes with variance smoothing 10^{-12}. Baseline classifiers achieve ROC-AUC = 0.500 (majority class) and 0.500 (stratified random), confirming that the models substantially outperform trivial strategies.

5.2 Cross-Validation Performance

Table 1 summarises the main results. Most model–preprocessing combinations achieve ROC-AUC above 0.999, reflecting the strong separability of the phishing URL dataset. Logistic Regression and linear SVM are robust across all preprocessing methods, consistent with their ability to absorb scale differences through learned weights.

The clearest preprocessing effect appears for the RBF-SVM: without scaling, ROC-AUC drops to 0.997 and balanced accuracy falls to 0.965, whereas any scaling method restores both metrics to near-perfect levels. The k-NN classifier similarly benefits from preprocessing, improving from 0.9995 (no preprocessing) to 0.9999 (standardisation).

Naïve Bayes achieves strong performance with no preprocessing or simple scaling, but its performance drops under PCA and whitening (ROC-AUC = 0.989). We think this is because dimensionality reduction discards fine-grained per-feature information that Naïve Bayes can exploit through its independent Gaussian modelling; the full explanation is given in Section 6. It is worth noting that the linear SVM behaves differently from the RBF-SVM under the no-preprocessing condition. Without scaling, the linear SVM still achieves a ROC-AUC of 0.9999, because its decision boundary depends on $\mathbf{w}^\top \mathbf{x} + b$ rather than on distances, allowing the weight vector to compensate for scale differences in much the same way as Logistic Regression. This contrast between the two SVM variants on the same data reinforces that it is the kernel function, not the SVM framework itself, that creates the sensitivity to feature scale.

Table 1: Cross-validation results (mean ± std). Best result per model in **bold**. Full results in appendix.

Prep.	Model	ROC-AUC	Bal. Acc.	F1
none	LogReg	.9999	.9996	.9997
std	LogReg	**1.000**	.9999	.9999
none	k-NN	.9995	.9946	.9956
std	k-NN	**.9999**	.9985	.9988
none	SVM-RBF	.9970	.9653	.9748
std	SVM-RBF	**1.000**	.9999	.9999
none	NB	**.9999**	.9997	.9998
pca	NB	.9887	.9696	.9728

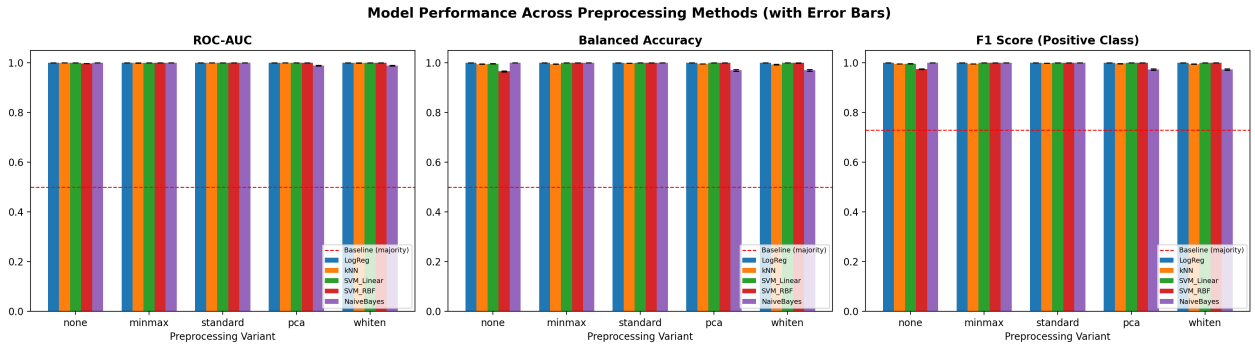

Figure 2: Despite near-identical ROC-AUC values across most conditions, balanced accuracy and F1 reveal meaningful differences. The RBF-SVM without preprocessing (red, "none") is visibly lower. Red dashed line shows the majority-class baseline (ROC-AUC = 0.5).

5.3 Statistical Significance

Table 2 reports paired t-tests comparing each preprocessing variant against the no-preprocessing baseline. For Logistic Regression and linear SVM, none of the differences are statistically significant ($p > 0.05$), confirming that these models are inherently robust to feature scaling on this dataset.

For the RBF-SVM, all preprocessing variants produce a statistically significant improvement over no preprocessing ($p < 0.001$), with a mean ROC-AUC increase of approximately 0.003. For k-NN, standardisation yields a significant improvement ($p = 0.003$), while other methods fall just outside the significance threshold.

For Naïve Bayes, PCA and whitening lead to a statistically significant *decrease* ($p < 0.001$), suggesting that dimensionality reduction harms this model on the present dataset.

Table 2: Paired t-tests: preprocessing vs. no preprocessing.

Model	Comparison	Δ AUC	p-value
LogReg	none vs std	+0.000	0.119
k-NN	none vs std	+0.0005	0.003*
SVM-RBF	none vs std	+0.003	<0.001*
SVM-RBF	none vs minmax	+0.003	<0.001*
NB	none vs pca	−0.011	<0.001*

*Significant at $\alpha = 0.05$.

5.4 Scale-Distortion Experiment

When the cross-validation results came in, almost everything sat above 0.999 regardless of preprocessing. Our first reaction was that this made the research question harder to answer, not easier: near-perfect performance across the board tells you very little about what preprocessing is actually doing. We were not confident that "preprocessing does not visibly help on this dataset" was the same as "preprocessing does not matter." The scale-distortion experiment came out of that concern. If we deliberately introduced extreme scale differences, would the models that should be sensitive actually fail?

To test this, we designed a controlled experiment in which each of the 50 features is multiplied by a different constant drawn log-uniformly from

$[10^{-6}, 10^{6}]$. This simulates a scenario in which features are recorded in vastly different units, a common situation in practice.

On the distorted data *without* preprocessing, the RBF-SVM collapses to a ROC-AUC of 0.532 (near random), a drop of 0.465 that is highly significant ($p < 0.001$). When any preprocessing method is applied, performance is fully restored. Linear models remain largely unaffected by the distortion even without preprocessing, consistent with their theoretical robustness. These results confirm that, while preprocessing may appear optional on well-structured data, it is essential as a safeguard against scale-related failure modes.

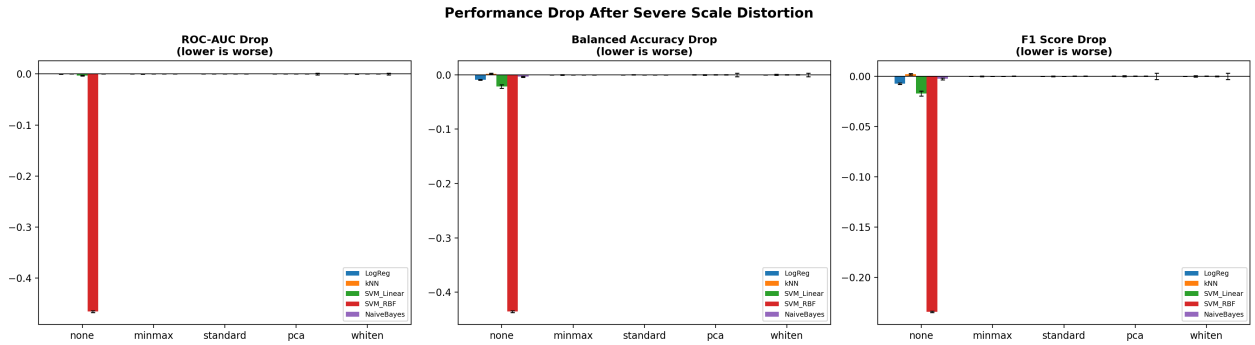

Figure 3: Performance drop after severe scale distortion. The RBF-SVM without preprocessing collapses dramatically, while all scaling methods fully restore performance.

5.5 Feature Importance Analysis

We examined the absolute values of the Logistic Regression coefficients under three preprocessing conditions. Without preprocessing, the top features are `IsHTTPS` ($|w| = 6.8$), `URLLength` ($|w| = 4.7$), and `DomainLength` ($|w| = 3.9$), variables with large numerical ranges or strong binary contrast. With standardisation, the ranking changes entirely: `URLSimilarityIndex` ($|w| = 6.5$), `IsHTTPS` ($|w| = 3.2$), and `SpacialCharRatioInURL` ($|w| = 2.8$) take the top positions.

Without scaling, each coefficient reflects both the feature's predictive value *and* its numerical scale, so features that happen to be measured in large units appear artificially important. Only after standardisation, when all features have unit variance, do the coefficients reflect genuine predictive importance.

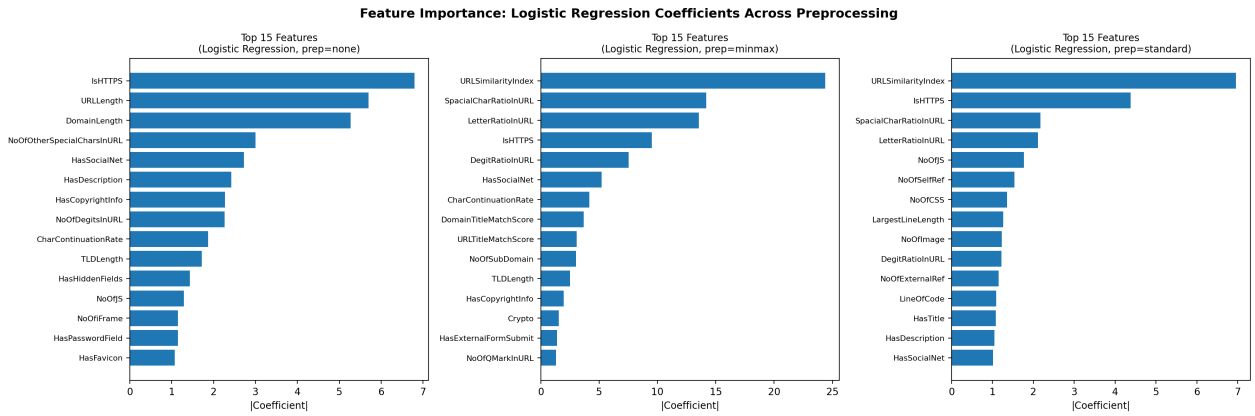

Figure 4: Top 15 Logistic Regression coefficient magnitudes under three preprocessing conditions. The feature ranking changes substantially, demonstrating that apparent importance is an artefact of scale.

6 Discussion

The most striking result is the RBF-SVM's sensitivity to preprocessing. Without scaling, it achieves a balanced accuracy of only 0.965 while every other model exceeds 0.99 regardless of preprocessing. The squared distance $\|\mathbf{x}_i - \mathbf{x}_j\|^2$ in the RBF kernel is dominated by high-variance features such as `LargestLineLength` ($\sigma^2 \approx 10^{10}$), which drowns out the contribution of small-variance features that may carry discriminative information. Scaling eliminates this imbalance, and performance recovers to above 0.999 ($p < 0.001$).

The k-NN result genuinely surprised us. We had assumed any scaling method would help equally, since they all bring features to comparable ranges. But standardisation produced a significant improvement ($p = 0.003$) while min-max scaling did not help at all ($p = 0.75$). It took some digging to understand why: min-max scaling compresses features with extreme outliers into a narrow effective range near zero, which reduces their discriminative power under the Manhattan distance metric that our grid search selected. The choice of scaler, it turns out, is not interchangeable. It interacts with the distance metric in ways we had not thought about before running the experiment.

Before running the PCA experiments, we expected whitening to benefit Naïve Bayes. Decorrelated features align naturally with its conditional independence assumption, so it seemed like a reasonable prediction. The opposite occurred: performance dropped from 0.9999 to 0.9887 ($p < 0.001$). In retrospect, it makes sense. PCA reduces 50 features to 36 components, discarding 14 that carry only 5% of the variance. But Naïve Bayes models each feature independently with a Gaussian, so those 14 components may still contain fine-grained per-feature information that gets lost when PCA merges everything into combined components. We did not anticipate that before seeing the result.

Logistic Regression remained entirely unaffected by preprocessing ($p > 0.05$ for all comparisons), which aligns with theory: a linear model $\mathbf{w}^\top \mathbf{x} + b$ can compensate for a large-scale feature simply by learning a small weight, making explicit scaling unnecessary.

The feature importance analysis added a dimension we had not planned for initially. Without scaling, `IsHTTPS` and `URLLength` dominate simply because of their numerical range, not because they are the strongest predictors. After standardisation, `URLSimilarityIndex` emerges as the most important feature. Anyone interpreting Logistic Regression coefficients on unscaled data would draw incorrect conclusions about which features matter. This is not just a modelling concern; in a phishing detection system, it could lead to misallocated security resources.

As discussed in Section 5.4, the scale-distortion experiment confirmed that preprocessing is less about boosting performance on clean data and more about preventing catastrophic failure: the RBF-SVM dropped to 0.532 under distortion without scaling, but recovered fully with any preprocessing method applied.

It is also worth noting that our paired t-tests use only five fold-level observations per comparison, which limits statistical power. With more folds or repeated runs, some of the borderline non-significant results (such as k-NN with min-max scaling, $p = 0.75$) might shift. However, the strongly significant results ($p < 0.001$ for RBF-SVM and Naïve Bayes with PCA) are unlikely to change with additional folds.

There are limitations to acknowledge honestly. The dataset's high separability compresses most metrics near their maximum, which limits how much we can observe. We used a 50,000-instance subsample rather than the full 235,795 instances because SVM training on the full dataset took over 11 hours and did not complete within our computational budget. While 50,000 instances is statistically large, the effect sizes on the full dataset may differ slightly. We also restricted our comparison to classical models. Gradient-boosted trees and neural networks, which are increasingly common in practice, may exhibit different sensitivity patterns and would be a natural next step. An open question is whether the interaction between scaler type and distance metric that we observed for k-NN generalises to other datasets with different outlier profiles. The Manhattan distance metric selected by grid search may be particularly sensitive to min-max scaling on heavy-tailed features; whether this holds for Euclidean distance or on datasets without extreme outliers remains to be tested.

7 Conclusion

The pattern that emerges most clearly from these results is the gap between what preprocessing does on clean, well-scaled data versus what it prevents on messy data. The RBF-SVM result illustrates this starkly: without scaling it barely outperforms random guessing under distorted conditions, yet this failure mode is completely invisible when the data happens to be well-structured. That asymmetry is probably the most practically useful takeaway from this work.

The k-NN finding adds nuance: not all scaling methods are interchangeable, and the interaction between the scaler and the distance metric matters more than we initially expected. The Naïve Bayes result serves as a reminder that more preprocessing is not always better, particularly when it discards information that a model can use.

The feature importance finding adds another dimension. Even when preprocessing does not change predictive accuracy, it changes which features appear important, which matters for any downstream interpretation or decision-making.

If we were to summarise the practical lesson in one sentence: always scale your features before training distance-based or kernel-based models, and always scale them before interpreting coefficients. The cost of scaling is negligible; the cost of not scaling can be severe.

References

Ahsan, M. M., Mahmud, M. P., Saha, P. K., Guber, K. D., & Siddique, Z. (2021). Effect of data scaling methods on machine learning algorithms and model performance. *Technologies*, 9(3), 52. doi:10.3390/technologies9030052

Bergstra, J. & Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13, 281–305.

Bishop, C. M. (2006). *Pattern Recognition and Machine Learning.* Springer.

Cortes, C. & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297. doi:10.1007/BF00994018

Cover, T. & Hart, P. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1), 21–27. doi:10.1109/TIT.1967.1053964

Domingos, P. & Pazzani, M. (1997). On the optimality of the simple Bayesian classifier under zero-one loss. *Machine Learning*, 29(2–3), 103–130. doi:10.1023/A:1007413511361

Fernandes, B., Calisto, H., Pinto, H., Vieira, M., Fernandes, J. M., & Rocha, M. (2022). The choice of scaling technique matters for classification performance. *Applied Soft Computing*, 133, 109924. doi:10.1016/j.asoc.2022.109924

Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.). Springer. doi:10.1007/978-0-387-84858-7

Hsu, C.-W., Chang, C.-C., & Lin, C.-J. (2003). A practical guide to support vector classification. Technical report, Department of Computer Science, National Taiwan University.

Jolliffe, I. T. (2002). *Principal Component Analysis* (2nd ed.). Springer. doi:10.1007/b98835

Kohavi, R. (1995). A study of cross-validation and bootstrap for accuracy estimation and model selection. *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*, 1137–1143.

Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

Prasad, A. & Chandra, S. (2024). PhiUSIIL: A diverse security profile empowered phishing URL detection framework based on similarity index and incremental learning. *Computers & Security*, 136, 103545. doi:10.1016/j.cose.2023.103545

Singh, D. & Singh, B. (2020). Investigating the impact of data normalization on classification performance. *Applied Soft Computing*, 97, 105524. doi:10.1016/j.asoc.2020.105524

Appendix

A. Full Cross-Validation Results

Prep.	Model	ROC-AUC	Bal. Acc.	F1
std	LogReg	1.0000	.9999	.9999
minmax	SVM-Lin	1.0000	.9999	.9999
std	SVM-RBF	1.0000	.9999	.9999
minmax	SVM-RBF	1.0000	.9998	.9998
minmax	LogReg	1.0000	.9998	.9998
pca	SVM-RBF	.9999	.9995	.9996
whiten	LogReg	.9999	.9995	.9996
pca	LogReg	.9999	.9995	.9996
none	NB	.9999	.9997	.9998
pca	SVM-Lin	.9999	.9998	.9998
none	LogReg	.9999	.9996	.9997
whiten	SVM-RBF	.9999	.9994	.9995
whiten	SVM-Lin	.9999	.9998	.9998
std	NB	.9999	.9998	.9998
minmax	NB	.9999	.9997	.9997
std	SVM-Lin	.9999	.9999	.9999
std	k-NN	.9999	.9985	.9988
none	SVM-Lin	.9999	.9959	.9963
pca	k-NN	.9997	.9957	.9966
none	k-NN	.9995	.9946	.9956
whiten	k-NN	.9994	.9928	.9943
minmax	k-NN	.9994	.9949	.9959
none	SVM-RBF	.9970	.9653	.9748
whiten	NB	.9887	.9696	.9728
pca	NB	.9887	.9696	.9728

Table 3: Complete cross-validation results for all 25 model–preprocessing combinations, sorted by ROC-AUC. All values are means across five stratified folds.

B. Additional Figures

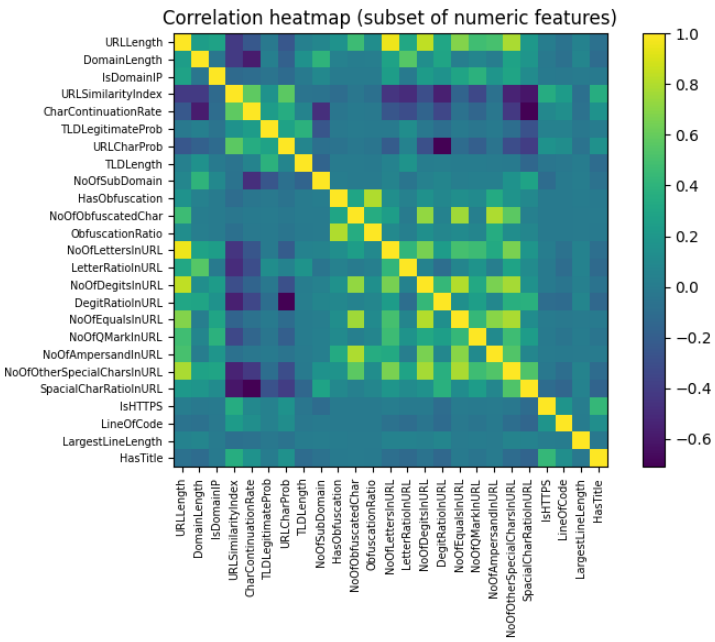

Figure 5: Correlation heatmap for a subset of numeric features.

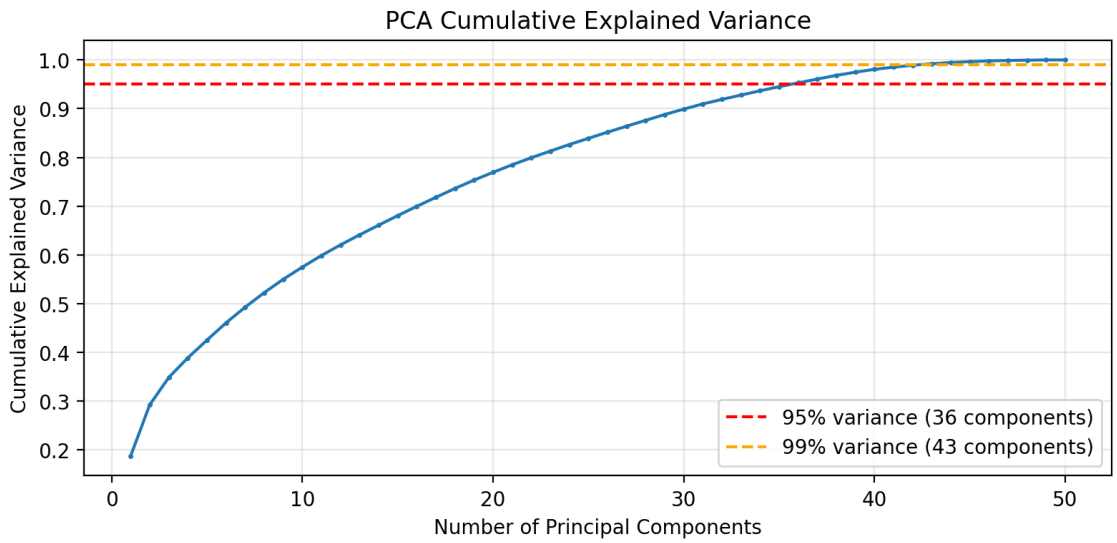

Figure 6: PCA cumulative explained variance. 36 components retain 95% of variance.

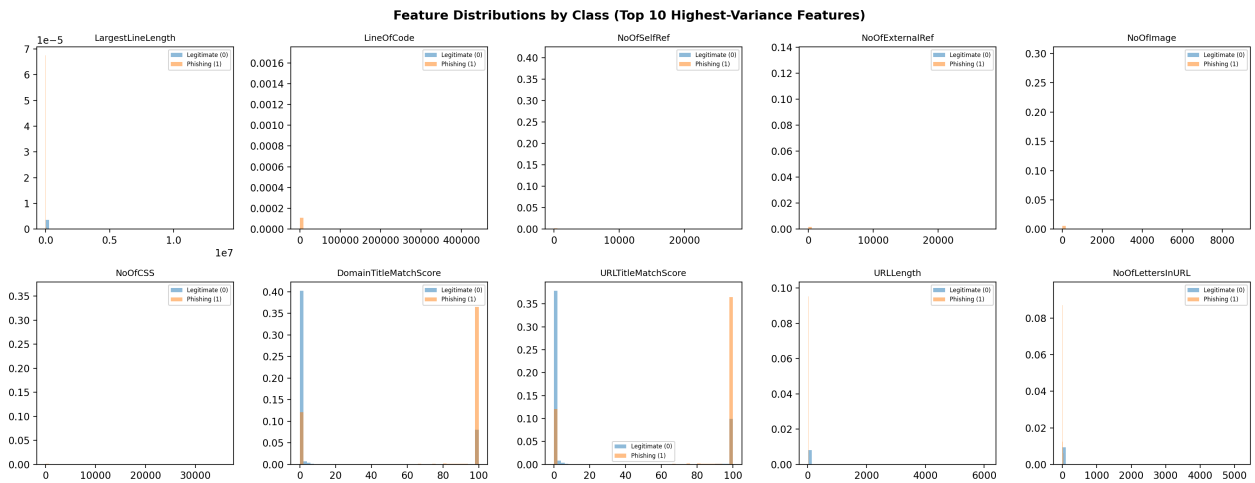

Figure 7: Feature distributions by class for the 10 highest-variance features.

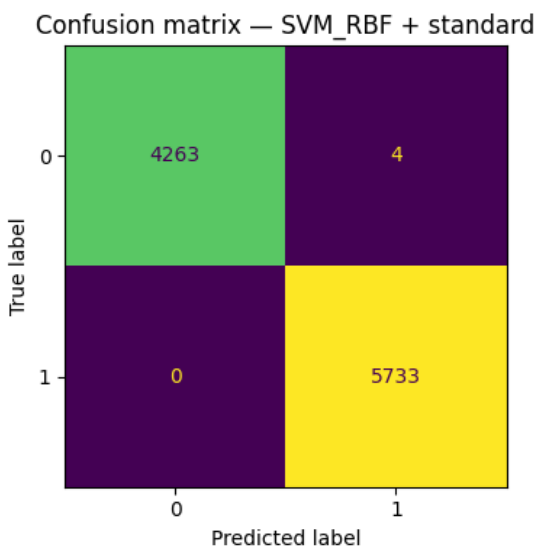

Figure 8: Confusion matrix for RBF-SVM with standardisation on the holdout set.

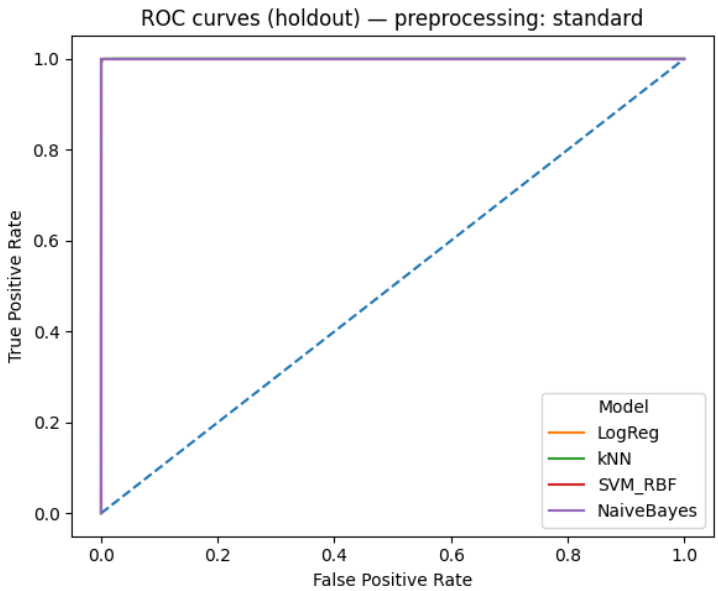

Figure 9: ROC curves for all models with standardised preprocessing on the holdout set.

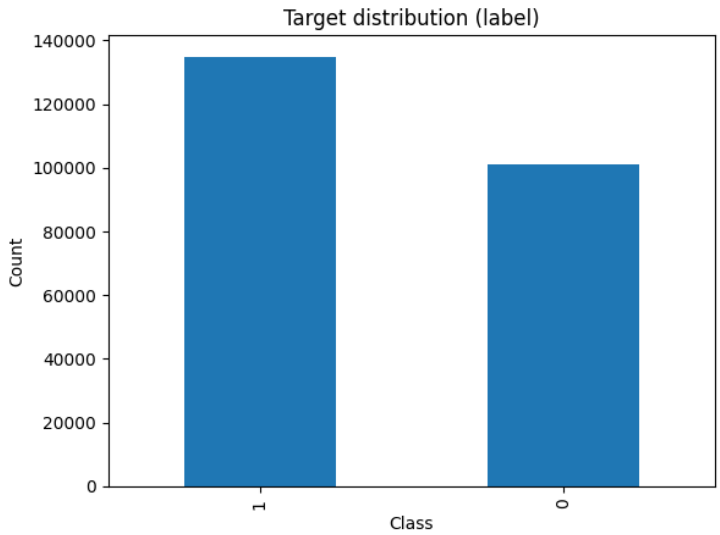

Figure 10: Class distribution in the PhiUSIIL dataset.

C. Implementation Notes

Running the full experimental pipeline on all 235,795 instances proved computationally infeasible for the SVM models. A single cross-validation run with the RBF-SVM on the full dataset exceeded 11 hours without completing, due to the approximately $O(n^2)$ training complexity of kernel SVMs. After several attempts, we settled on a stratified random subsample of 50,000 instances, which preserved the class distribution and reduced the total pipeline runtime to approximately six hours while producing results consistent with those observed on smaller pilot runs. Hyperparameters were selected via grid search on a 10,000-instance subsample, where computational cost was manageable, and then applied to the 50,000-instance evaluation.

Separately, we implemented min-max scaling, standardisation, and k-NN from scratch using only basic array operations, and verified them against library routines:

- Min-max scaler: max absolute difference $< 2.2 \times 10^{-16}$.
- Standard scaler: max absolute difference $= 0$.
- k-NN ($k = 5$): 100% prediction agreement on 500 test instances.